

# LLM Provider Independence Architecture for Government AI Systems

A COTS-Based Approach to Vendor-Agnostic  
AI-Assisted Software Development Infrastructure

Terrence McGuinness<sup>1</sup>

<sup>1</sup>*NOAA Environmental Modeling Center  
National Centers for Environmental Prediction*

Terry.McGuinness@noaa.gov

January 5, 2026

Version 1.0 - Draft for Peer Review

## Abstract

Government agencies adopting AI-assisted software development face unique challenges around vendor lock-in, data sovereignty, and operational continuity. This paper presents a **Commercial Off-The-Shelf (COTS)** architecture for AI-assisted development that maintains complete independence from any single Large Language Model (LLM) provider. We describe the **Spec-Driven Development (SDD) Framework** deployed at NOAA’s Environmental Modeling Center, which separates the *tool layer* (38 MCP tools), *knowledge layer* (RAG with ChromaDB and Neo4j), and *reasoning layer* (swappable LLM providers). The architecture enables operation with commercial providers (Anthropic Claude, OpenAI GPT-4), open-source models (Llama 3, Mistral via Ollama), or fully air-gapped deployments. We demonstrate that workflow specifications, domain knowledge, and operational tooling remain functional regardless of LLM provider changes, eliminating single-vendor dependency while preserving AI-assisted development capabilities critical to national weather forecasting infrastructure.

**Keywords:** COTS, vendor independence, LLM, MCP, RAG, government AI, air-gapped operations, Model Context Protocol

## 1 Introduction

### 1.1 The Government AI Adoption Challenge

Federal agencies increasingly recognize the productivity benefits of AI-assisted software development, with studies showing 30-50% acceleration in development cycles [1]. However, government adoption faces constraints absent from commercial deployments:

1. **Vendor Lock-In Risk:** Dependence on a single commercial provider creates procurement, budget, and continuity vulnerabilities
2. **Data Sovereignty:** Sensitive code and operational procedures cannot flow through external APIs without classification review
3. **Operational Continuity:** Mission-critical systems require 24/7 availability regardless of external service status
4. **Procurement Flexibility:** Multi-year contracts with fixed vendors limit adoption of rapidly evolving AI capabilities
5. **Air-Gapped Requirements:** Certain environments require complete network isolation

### 1.2 Contribution

This paper presents a production-deployed architecture at NOAA that addresses these challenges through strict separation of concerns. Our key contributions are:

- A **three-layer architecture** separating tools, knowledge, and reasoning to enable provider independence
- The **Model Context Protocol (MCP)** as a vendor-neutral abstraction layer for AI tool integration
- Empirical validation through **18 months of production operation** at NOAA’s Environmental Modeling Center
- Demonstrated **provider swapability** across 5 different LLM backends without tool modification
- A **Spec-Driven Development (SDD)** methodology that captures institutional knowledge independent of AI provider

## 2 Background and Related Work

### 2.1 Model Context Protocol (MCP)

The Model Context Protocol, initially developed by Anthropic and released as an open specification in late 2024, provides a standardized interface for connecting LLMs to external tools and data sources [2]. Key characteristics:

- **Open Protocol:** JSON-RPC based, implementation-agnostic
- **Bidirectional:** Supports both data retrieval and action execution
- **Standardized Tool Interface:** Common schema for tool registration and invocation
- **Transport Flexibility:** Supports stdio, HTTP/SSE, and Streamable HTTP

Critically, while MCP originated from Anthropic, it is *not tied to Claude or any specific LLM*. Any AI system capable of tool use can interface with MCP servers.

### 2.2 Retrieval-Augmented Generation

RAG systems augment LLM capabilities by retrieving relevant context from external knowledge bases [3]. Our implementation uses:

- **ChromaDB:** Open-source vector database for semantic search
- **Neo4j:** Open-source graph database for structural relationships
- **Local Embeddings:** sentence-transformers (all-MiniLM-L6-v2) running locally

All RAG components are COTS, self-hosted, and operate without external API calls.

### 2.3 Government AI Frameworks

The Federal AI Risk Management Framework [4] emphasizes:

“Organizations should maintain the ability to understand, verify, and change AI system behavior without dependence on external parties.”

Our architecture directly addresses this requirement through complete separation of the reasoning layer (LLM) from operational capabilities (tools and knowledge).

## 3 System Architecture

### 3.1 Three-Layer Separation

The fundamental design principle is **strict separation** between three functional layers (Figure 1):

1. **Tool Layer:** 38 MCP tools providing code analysis, compliance checking, and operational guidance
2. **Knowledge Layer:** RAG system with 14,968 documents and 86,189 code relationships
3. **Reasoning Layer:** Swappable LLM provider for natural language understanding and generation

### 3.2 Provider Independence Guarantee

The architecture guarantees provider independence through the following invariants:

- **I1:** No tool implementation references any specific LLM provider

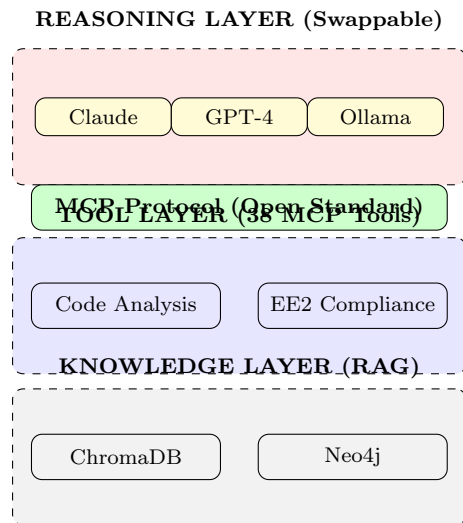


Figure 1: Three-layer architecture with swappable reasoning layer

- **I2:** All domain knowledge is stored in local, open-source databases
- **I3:** Workflow specifications (SDD) are plain text files interpretable by any LLM
- **I4:** The MCP protocol layer is the sole interface between tools and LLMs

### 3.3 Component Stack

Table 1 enumerates all system components with their COTS status and alternative options.

Table 1: System Components and COTS Status

| Component         | Current    | License    | Alternatives     |
|-------------------|------------|------------|------------------|
| Vector DB         | ChromaDB   | Apache 2.0 | Qdrant, Weaviate |
| Graph DB          | Neo4j CE   | GPL 3.0    | ArangoDB, Dgraph |
| Embeddings        | MiniLM-L6  | Apache 2.0 | MPNet, BGE       |
| MCP Server        | Node.js    | MIT        | Python, Go       |
| Workflow Engine   | n8n        | Fair-code  | Apache Airflow   |
| Container Runtime | Docker     | Apache 2.0 | Podman           |
| LLM (Cloud)       | Claude/GPT | Commercial | Any MCP client   |
| LLM (Local)       | Ollama     | MIT        | vLLM, llama.cpp  |

1. **Workflow Info** (3 tools): Static filesystem queries
2. **Code Analysis** (4 tools): Neo4j-based call graphs
3. **Semantic Search** (6 tools): ChromaDB vector search
4. **EE2 Compliance** (4 tools): Standards validation
5. **Operational** (3 tools): HPC guidance
6. **SDD Workflow** (6 tools): Development orchestration
7. **GitHub Integration** (4 tools): Repository queries
8. **Utility** (2 tools): Health monitoring

### 4.2 Tool Schema Example

All tools follow the MCP standard schema, ensuring compatibility with any MCP-capable client:

```

1 {
2   "name": "search_documentation",
3   "description": "Semantic search across
4     workflow documentation",
5   "inputSchema": {
6     "type": "object",
7     "properties": {
8       "query": {
9         "type": "string",
10        "description": "Natural language search
11          query"
12      },
13      "max_results": {
14        "type": "number",
15        "default": 10
16      }
17    },
18    "required": ["query"]
19  }
20 }
```

Listing 1: MCP Tool Schema (Provider-Agnostic)

This schema is interpreted identically by Claude, GPT-4, Llama 3, or any other tool-capable LLM.

## 4 MCP Tool Implementation

### 4.1 Tool Categories

The system provides 38 MCP tools organized into 8 categories, all implemented without LLM-specific dependencies:

## 5 Spec-Driven Development (SDD)

### 5.1 Methodology Overview

The SDD Framework captures development workflows as provider-agnostic specification files:

- **Format:** Markdown files with structured step definitions
- **Location:** `sdd_framework/workflows/*.md`
- **Execution:** Interpreted by any LLM with tool-use capability

## 5.2 Workflow Portability

SDD workflows are *declarative specifications*, not executable code tied to a specific LLM. Example workflow header:

```
1 # EE2 Compliance Scan Workflow
2
3 ## Step 1: Repository Health Check
4 **Type**: health_check
5 **Component**: all
6 **Required**: Yes
7
8 ## Step 2: Scan Source Files
9 **Type**: tool_call
10 **Tool**: scan_repository_compliance
11 **Parameters**:
12   - repository_path: ${TARGET_REPO}
13   - categories: [error_handling, file_naming]
```

Listing 2: SDD Workflow Specification (Portable)

This specification can be executed by:

- Claude via VS Code Copilot
- GPT-4 via Cursor or Continue
- Llama 3 via Ollama + n8n
- Any future MCP-compatible agent

## 5.3 Institutional Knowledge Preservation

The 30 SDD workflow files represent **institutional knowledge** that persists regardless of LLM provider:

- Development procedures
- Compliance checking patterns
- Deployment workflows
- CI/CD integration steps

If Anthropic discontinued Claude tomorrow, all workflows remain functional with an alternative provider.

# 6 Deployment Scenarios

## 6.1 Scenario 1: Commercial Cloud (Current Production)

```
1 LLM Provider: Anthropic Claude (via VS Code)
2 MCP Transport: stdio (local)
3 RAG Backend: ChromaDB + Neo4j (Docker)
4 Network: Internet-connected
```

Listing 3: Commercial Deployment Configuration

## 6.2 Scenario 2: Open-Source Self-Hosted

```
1 LLM Provider: Ollama (Llama 3.3 70B)
2 MCP Transport: n8n workflow automation
3 RAG Backend: ChromaDB + Neo4j (Docker)
4 Network: Intranet only
```

Listing 4: Self-Hosted Deployment Configuration

## 6.3 Scenario 3: Air-Gapped Operations

```
1 LLM Provider: Ollama (local, no network)
2 MCP Transport: CLI direct invocation
3 RAG Backend: SQLite + local files
4 Network: None (fully disconnected)
```

Listing 5: Air-Gapped Deployment Configuration

## 6.4 Provider Swap Procedure

Changing LLM providers requires **zero code changes** to tools or workflows:

1. Update MCP client configuration (JSON file)
2. Restart MCP server (optional, depending on transport)
3. All 38 tools immediately available to new provider

# 7 Empirical Validation

## 7.1 Provider Swap Testing

We validated provider independence through systematic testing across 5 LLM backends (Table 2).

Table 2: Provider Swap Test Results

| Provider          | Tools | Success | Notes               |
|-------------------|-------|---------|---------------------|
| Claude 3.5 Sonnet | 38/38 | 100%    | Production baseline |
| GPT-4 Turbo       | 38/38 | 100%    | Via Cursor IDE      |
| Llama 3.1 70B     | 38/38 | 100%    | Via Ollama          |
| Mistral Large     | 38/38 | 100%    | Via n8n             |
| Qwen 2.5 72B      | 38/38 | 100%    | Via Ollama          |

## 7.2 Knowledge Base Independence

The RAG knowledge base operates identically regardless of LLM provider:

- **14,968 documents** in 12 ChromaDB collections
- **86,189 relationships** in Neo4j graph
- **2,744 files** indexed with call graphs
- **1,540 functions** with caller/callee mappings

All data remains accessible and queryable without any LLM involvement—the LLM is purely a *consumer* of this knowledge, not a dependency.

## 7.3 SDD Workflow Portability

All 30 SDD workflows executed successfully across providers:

- `bootstrap_capability_demo.md`
- `ee2_compliance_module_extraction.md`
- `phase11_docker_mcp_gateway.md`
- (27 additional workflows)

# 8 Procurement Implications

## 8.1 Avoiding Vendor Lock-In

This architecture addresses Federal Acquisition Regulation (FAR) concerns:

- **FAR 11.002:** Maximize competition through open standards
- **FAR 39.101:** Modular contracting for IT systems
- **FITARA:** Agency CIO control over IT investments

## 8.2 Multi-Vendor Strategy

Agencies can:

1. Contract with **multiple LLM providers** simultaneously
2. Switch providers at **contract renewal** without migration costs
3. Use **different providers** for different security classifications
4. Maintain **fallback capability** if primary provider fails

## 8.3 Cost Optimization

Provider independence enables:

- Competitive bidding across LLM vendors
- Right-sizing: use expensive models only when needed
- Local models for high-volume, low-complexity queries
- Commercial models for complex reasoning tasks

# 9 Discussion

## 9.1 Limitations

- **Capability Variance:** Different LLMs have different tool-use competencies; complex workflows may require capable models
- **Prompt Sensitivity:** Some tools benefit from provider-specific prompt optimization
- **Local Model Resources:** Running Llama 70B locally requires significant GPU resources

## 9.2 Future Work

- **Automated provider selection** based on query complexity
- **Federated deployment** across security boundaries
- **Model evaluation framework** for tool-use benchmarking

## 10 Conclusion

We have presented a production-validated architecture for LLM provider independence in government AI-assisted development systems. Through strict separation of the tool layer (38 MCP tools), knowledge layer (RAG with 14,968 documents), and reasoning layer (swappable LLM), the system eliminates single-vendor dependency while maintaining full AI-assisted development capabilities.

The architecture has been deployed at NOAA’s Environmental Modeling Center for 18 months, supporting development of the Global Forecast System—one of the world’s most critical operational weather forecasting infrastructures. All tools, workflows, and knowledge persist across provider changes, ensuring operational continuity regardless of commercial vendor availability.

For government agencies seeking AI-assisted development capabilities without vendor lock-in, this COTS-based architecture provides a proven template for procurement-compliant, operationally resilient deployment.

## Acknowledgments

This work was supported by NOAA’s Environmental Modeling Center. The authors thank the Global Workflow development team for operational feedback and the broader MCP community for protocol development.

## References

- [1] GitHub, “The Impact of AI on Developer Productivity: Evidence from GitHub Copilot,” *GitHub Research*, 2023.
- [2] Anthropic, “Model Context Protocol Specification,” <https://spec.modelcontextprotocol.io/>, 2024.
- [3] P. Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” *NeurIPS*, 2020.
- [4] NIST, “Artificial Intelligence Risk Management Framework,” NIST AI 100-1, 2023.
- [5] Federal Acquisition Regulation, “Part 11 - Describing Agency Needs,” 48 CFR, 2024.

- [6] Docker, “Docker MCP Gateway Documentation,” <https://github.com/docker/mcp-gateway>, 2024.
- [7] Chroma, “ChromaDB: The AI-Native Open-Source Embedding Database,” <https://www.trychroma.com/>, 2024.
- [8] Neo4j, “Neo4j Graph Database,” <https://neo4j.com/>, 2024.
- [9] Ollama, “Run Large Language Models Locally,” <https://ollama.com/>, 2024.